

1. Data structure

A structure is a value type data type. It helps you to make a single variable hold related data of various data types. The struct keyword is used for creating a structure.

2. Class versus Structure

Classes and Structures have the following basic differences "
classes are reference types and structs are value types
structures do not support inheritance
structures cannot have default constructor

3. Why C# does not support multiple inheritance

C# does not support multiple inheritance because it might raise the issue of ambiguity.

The ambiguity arises when two parent classes have the same child class, and each parent class has a function with the same name.

4. C# interface

The interface is a class in C#,
which helps us implement the C# multiple inheritance.
In the interface, we only define the method, not the implementation of the method.
We can overwrite the implementation of the method when we inherit the interface's functionality in the derived class. Interface supports pure abstraction. An interface cannot contain variables, constant members, constructors, destructors, or static members.

5. Design Patterns

Design patterns are solutions to software design problems you find again & again in real-world application development.

Singleton -> The Singleton design pattern ensures a class has only one instance and provide a global point of access to it.

Factory Method -> Abstract class with abstract method

Abstract Factory -> create abstract class with related abstract methods or families

6. Collection

Dictionary<TKey,TValue>--> Store items as key/value pairs for quick look-up by

key

List<T> --> Access items by index

LinkedList<T> --> Access items sequentially

SortedList<TKey,TValue> --> A sorted collection

HashSet<T>, SortedSet<T> --> A set for mathematical functions

generic -> it is

Weight

100

200

300

400

To. 1000

7. Hashtable Vs Dictionary

Dictionary, store key/value pairs of same type. --> Hashtable, store key/value pairs of the same or different type.

Dictionary, must specify the type of key and value. --> Hashtable, no need to specify the type of the key and value.

Dictionary, data retrieval is faster --> Hashtable, data retrieval is slower

Dictionary, give error if key is not available when fetch --> HashSet, give null if key is not available when fetch

8. Caches

It essentially stores information that is likely to be used again.

9. OOPs Concepts

What is OOP?

OOP (Object-Oriented Programming) is a programming paradigm where you design your program using objects — that is, entities that contain data (attributes) and behavior (methods).

Abstraction --> Abstraction means putting all the necessary variables and methods in a class.

Encapsulation --> Wrapping up a data member and a method together into a single unit [Class]

Inheritance --> When a class includes a property of another class, it is known as inheritance

Polymorphism --> Polymorphism means one name, many forms. One function behaves in different forms.

Concept Why It's Important
Encapsulation Protects data
Inheritance Reuses code
Polymorphism Increases flexibility
Abstraction Simplifies complexity

why OOPs require in our project

Benefit | Explanation

----- |

-
1. Code Reusability | You can reuse classes and objects instead of writing the same logic again. For example, if you make a `User` class, you can reuse it across different parts of your app.
 2. Easy Maintenance | If something changes, you only update the relevant class — no need to edit all the code.
 3. Modularity | The project is divided into small, independent pieces (objects/classes), making it easier to understand and manage.
 4. Scalability | As the project grows, OOP helps you extend functionality without breaking existing code.
 5. Encapsulation | Keeps data safe by hiding internal details and only exposing what's necessary (like a remote control).
 6. Abstraction | Focus on what an object does, not how it does it — improves readability.
 7. Inheritance | Share common logic between related classes (e.g., `Admin` inherits from `User`).
 8. Polymorphism | Write flexible code that can work with multiple object types — especially useful in large systems.

10. Entity framework

Use to save or retrieve application data from a database.

Cross-platform --> Used in Windows, Linux, MAC

Linq Queries --> Can use linq queries

Saving --> SaveChanges() OR SaveChangesAsync() --> Insert, update, delete

Caching --> repeated querying will return data from the cache instead of hitting the database.

Migrations --> Provides set of migration commands that can be executed on the NuGet Package Manager Console or the Command Line Interface to create/manage database Schema.

10. What is Entity Framework (EF)?

Entity Framework is an Object-Relational Mapper (ORM) for .NET.
It allows developers to work with databases using C# objects instead of writing SQL queries directly.

11. Synchronous & Asynchronous

Synchronous --> Once one operation is complete then another will run
Asynchronous --> One operation execution is not dependent on another, Multiple operation can run on single time

12. Loose Coupling & Tight Coupling

Tight Coupling means one class is dependent on another class.
In tight coupling, there are hard-coded dependency declared in methods.
Loose Coupling means one class is dependent on interface rather than class.
In loose coupling, we must pass dependency externally at runtime instead of hard-coded.

13. Can the .tostring method handle NULL values?

No, the .tostring method cannot handle NULL values. Instead of it, we can use Convert .tostring method to handle NULL values.

14. Angular

Today 1st Interview Question (27 Jun 2023)

1. Is .net core is better then .net framework, If yes how? and their differences
2. Dependency injection
3. Metapackage - package.lock.json
4. Middleware
5. Is web.config is available in .net core?
6. which files are available with .json extension in .net core?
7. Razor pages in .net core
8. How can we define services in controller?
9. Store procedure benefits
10. Normalization
11. Indexing in DB and Its types

12. Can we deploy .net core project in Linux platform?

13. CICD Approach

Today 2nd Interview Question (27 Jun 2023)

1. Write a sorting algorithm on a array without the use of any inbuilt function or tool, Sort the array in ascending order and take the input to be integer array

=(1,-2,3,8,-2)

2. Consider a hypothetical situation where you have a database of the number of employees in an organization with their salary, Write a linq expression to find out the record with maximum salary

3. Write an sql query to fetch the item name with the 2nd lowest value when compared to the average of the amount in the orders table

4. Difference between Add scoped and Add transient

5. Reflection in .net

6. Generic in .net

Programming Questions

1. Write a sorting algorithm on a array without the use of any inbuilt function or tool, Sort the array in ascending order and take the input to be integer array

=(1,-2,3,8,-2)

`int[] array = new int[] { 1, -2, 3, 8, -2 };`

`// Bubble Sort`

`int temp;`

`for (int i = 0; i < array.Length - 1; i++)`

`{`

`for (int j = 0; j < array.Length - i - 1; j++)`

`{`

`if (array[j] > array[j + 1])`

`{`

`// Swap elements`

`temp = array[j];`

`array[j] = array[j + 1];`

`array[j + 1] = temp;`

`}`

`}`

`}`

`// Print the sorted array`

`foreach (int num in array)`

`{`

`Console.Write(num + " ");`

`}`

2. Consider a hypothetical situation where you have a database of the number of employees in an organization with their salary, Write a linq expression to find out the record with maximum salary

```
var employeeWithMaxSalary = employees.OrderByDescending(e =>
e.Salary).FirstOrDefault();
```

3. Write an sql query to fetch the item name with the 2nd lowest value when compared to the average of the amount in the orders table
select avg(amount) as averageAmount from orders

Boxing & Unboxing

It is a concept to convert .net data types from value type to reference type AND vice versa

EX:

```
int i = 1;
```

```
object box = i; //Boxing
```

```
int j = (int)box; //Unboxing
```

readonly & constant

constant need to be defined at the time of assignment/declaration, while readonly field can be defined at runtime

constant are implicitly static, while readonly values don't have to be static they can change on runtime

Private constructor

Private constructor is a special instance constructor which is used in a class that contains static member only.

If a class has one or more private constructor then class cannot be inherited, cannot create object of a class.

It is used in singleton design patterns, to make sure that the only one instance of a class can ever be created.

Sealed class

Sealed classes are used to restrict the users from inheriting the class

Static class

Static class is a class that cannot be instantiated and cannot be inherited.

Static class can only contain static data members, static methods.

Static class cannot contain an instance constructor. However, they can contain a static constructor to initialize static members.

Abstract class (Global)

Abstract class is a class that cannot be instantiated. Instead, It serves as a base class for other classes to inherit from.

Abstract class can inherit from a class and one or more interfaces.

Abstract class can have modifiers for methods, properties, constants and fields, constructors or destructors.

Abstract class cannot be inherited from by structures

interface

The interface is a concept in C#, which helps us implement the C# multiple inheritance.

In the interface, we only define the method, not the implementation of the method.

We can overwrite the implementation of the method when we inherit the interface's functionality in the derived class. Interface supports pure abstraction.

An interface cannot contain variables, constant members, constructors, destructors, or static members.

Abstract class & interface

A class can implement any number of interfaces but a subclass can at most use only one abstract class.

An abstract class can have non-abstract Methods(concrete methods) while in case of Interface all the methods has to be abstract.

An abstract class can declare or use any variables while an interface is not allowed to do so.

An abstract class can have constructor declaration while an interface can not do so.

An abstract Class is allowed to have all access modifiers for all of its member declaration while in interface we cannot use access modifier(including public) as all the members of interface are implicitly public.

Delegates

The delegate is a reference type data type that defines the method signature.
A delegate is a pointer to a method. That means, a delegate holds the address of a method which can be called using that delegate

Generics

Generic is a class which allows the user to define classes and methods with the placeholder

The basic idea behind using Generic is to allow type (Integer, String, ... etc and user-defined types) to be a parameter to methods, classes, and interfaces.

EX:

Dictionary<TKey,TValue> --> Store items as key/value pairs for quick look-up by key

List<T> --> Access items by index

LinkedList<T> --> Access items sequentially

SortedList<TKey,TValue> --> A sorted collection

HashSet<T>, SortedSet<T> --> A set for mathematical functions

Extension Method

A C# extension method allows developers to extend the functionality of an existing type without creating a new derived type, recompiling, or otherwise modifying the original type.

Extension methods allow existing classes to be extended without relying on inheritance or changing the class's source code

An extension method must be defined in a top-level static class.

The concept of extension methods cannot be applied to fields, properties, or events.

ref vs out

Ref and out keywords in C# are used to pass arguments within a method or function
Both indicate that an argument/parameter is passed by reference. By default parameters are passed to a method by value

The parameter or argument must be initialized first before it is passed to ref.

It is not compulsory to initialize a parameter or argument before it is passed to an out.

Passing a parameter value by Ref is useful when the called method is also needed to modify the pass parameter.

Declaring a parameter to an out method is useful when multiple values need to be returned from a function or method.

using statement

The using statement is used to work with an object in C# that implements the IDisposable interface, where object is dispose after using block is closed.

var result = context.persons.where(p => p.id > 30).OrderBy(p => p.name, p.location)
Reports module not working or have not access, for that discussed with Michael Shewan and fixed to connect with Anand also.

Task

NET framework provides Threading.Tasks class to let you create tasks and run them asynchronously.

A task is an object that represents some work that should be done.

The task can tell you if the work is completed and if the operation returns a result, the task gives you the result.

Thread

.NET Framework has thread-associated classes in System.Threading namespace. A Thread is a small set of executable instructions.

Jira board :

Jira is used by a large number of clients and users globally for project, time, requirements, task, bug, change, code, test, release, sprint management.

Sprint Management :

Sprint is a time-boxed period during which a team works on a specific set of objectives, benchmarks, or outputs.

Confluence :

Confluence is a team workspace where knowledge and collaboration meet. Dynamic pages give your team a place to create, capture, and collaborate on any project or idea.

Concrete Class :

A concrete class is a class that we can create an instance of, using the new keyword.

In other words, it's a full implementation of its blueprint.

Stateless Protocol vs Stateful Protocol

Stateless Protocol:

-
1. does not require the server to retain the server information or session details.
 2. there is no tight dependency between server and client.
 3. The Stateless protocol design simplify the server design.
 4. Stateless Protocols handle the transaction very fastly.
 5. Stateless Protocols are easy to implement in Internet.
 6. Scaling architecture is relatively easier.
 7. The requests are not dependent on the server side and are self contained.
 8. To process different information at a time , different servers can be used.
 9. Example of Stateless are UDP , DNS , HTTP , etc.

Stateful Protocol:

-
1. Stateful Protocol require server to save the status and session information.
 2. In Stateful protocol, there is tight dependency between server and client
 3. The Stateful protocol design makes the design of server very complex and heavy.
 4. Stateful Protocols handle the transaction very slowly.
 5. Stateful protocols are logically heavy to implement in Internet.
 6. It is difficult and complex to scale architecture.
 7. The requests are always dependent on the server side.
 8. To process every request , the same server must be utilized.
 9. Example of Stateful are FTP , Telnet , etc.

Solid Principles :

-
- S — Single Responsibility: One class, one responsibility.
 - O — Open-Closed: Extensible, not modifiable.
 - L — Liskov Substitution: Substitute with subtypes.
 - I — Interface Segregation: Smaller, specific interfaces.
 - D — Dependency Inversion: Depend on abstractions, not concretions

Unit Testing :

test the smallest functional unit of code.

Unit testing involves testing individual parts of our code, like methods or classes, separately to make sure they work correctly.

By doing this, we can find and fix bugs, avoid problems, and create a strong codebase.

In .NET Core, there are three popular unit testing frameworks: MSTest, NUnit, and xUnit.net.

unit testing, "AAA" refers to the Arrange-Act-Assert pattern. This is a widely used structure for writing unit tests,

dividing each test method into three distinct phases: preparing the test environment (Arrange), invoking the code under test (Act), and verifying the outcome (Assert).

Integration Testing:

integration testing means end to end flow , individual units are combined and tested as a group

View vs StoreProcedure :

A Stored Procedure:

Accepts parameters

Can NOT be used as building block in a larger query

Can contain several statements, loops, IF ELSE, etc.

Can perform modifications to one or several tables

Can NOT be used as the target of an INSERT, UPDATE or DELETE statement.

A View:

Does NOT accept parameters

Can be used as building block in a larger query

Can contain only one single SELECT query

Can NOT perform modifications to any table

But can (sometimes) be used as the target of an INSERT, UPDATE or DELETE statement.

VARCHAR vs NVARCHAR :

VARCHAR uses a single-byte character set, meaning it can store characters from a single language or script.

NVARCHAR uses Unicode, allowing it to store characters from multiple languages and scripts

An array is a collection of fixed-size elements that are all of the same type, stored in contiguous memory.

What is .NET SDK?

The .NET SDK (Software Development Kit) is a collection of tools and libraries that you need to build, run, and develop .NET applications.

Request Delegation

The process of passing control from one middleware to the next in the pipeline using `await next()`.

MVC Filters :

1. Authentication Filters:

These filters are executed first and are used to authenticate the user's identity. They determine if the user is who they claim to be before further processing the request.

2. Authorization Filters:

These filters determine if the authenticated user has permission to access the requested resource.

They evaluate the user's roles and permissions against the requirements of the action being invoked.

3. Action Filters:

These filters run before and after an action method is executed.

They can be used for tasks like input validation, data transformation, or caching results.

4. Result Filters:

These filters run after an action method has executed, but before the result is rendered to the user.

They can be used to modify the view data or the response itself.

5. Exception Filters:

These filters are triggered when an unhandled exception occurs during the request processing.

They allow you to handle errors gracefully, log errors, or redirect the user to an error page.

What is Middleware in ASP.NET Core?

Middleware is software that's assembled into an app pipeline to handle requests and responses.

Each component:

Can perform actions before and after the next middleware.

Example: Logging, Authentication, Exception handling.

What is Dependency Injection in .NET? What are AddScoped, AddTransient, AddSingleton?

Dependency Injection (DI) is a design pattern to achieve Inversion of Control (IoC) for managing object lifetimes and dependencies.

AddScoped: New instance per request.

AddTransient: New instance every time requested.

AddSingleton: One instance for the whole app.

How does Exception Handling work in .NET Core?

Handled using try-catch-finally. For global exceptions, use middleware like `UseExceptionHandler()` or `DeveloperExceptionPage()`.

```
try
{
    // risky code
}
catch (Exception ex)
{
    // handle exception
}
```

What is Authentication & Authorization in .NET Core?

Authentication confirms who the user is (login).

Authorization checks what the user is allowed to do (role-based access).

Implemented using Identity or JWT in .NET Core.

What are Microservices?

Microservices are a software architecture pattern where an app is composed of small independent services communicating over HTTP

(often via REST APIs). Each service can be deployed and scaled independently.

What is CORS Policy in ASP.NET Core?

Cross-Origin Resource Sharing (CORS) allows or restricts resources to be requested from a different domain.

```
services.AddCors(options =>
{
options.AddPolicy("AllowAll", builder =>
builder.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader());
});
```

What are Extension Methods in C#?

Allows adding new methods to existing types without modifying them.

What is Entity Framework?

EF Core is an ORM (Object Relational Mapper) that lets you interact with databases using .NET objects.

```
var users = context.Users.ToList();
```

What is LINQ in C#?

Language Integrated Query (LINQ) is used for querying collections like arrays, lists, or databases in a type-safe way.

```
var adults = users.Where(u => u.Age >= 18);
```

What are Unit Test and Integration Test Cases?

Unit Test: Tests individual functions/classes (e.g., with xUnit or NUnit).

Integration Test: Tests multiple components working together (like DB + API).

What are HTTP Verbs (GET, POST, PUT, PATCH)?

GET: Fetch data

POST: Create resource

PUT: Update entire resource

PATCH: Update part of a resource

What is the first step to debug a production issue?

Check logs (e.g., Serilog, Application Insights).

Identify recent deployments or changes.

Replicate issue in staging.

Analyze performance metrics.

Database

What is the difference between Clustered and Non-Clustered Index?

Clustered Index: Alters the physical order of data. One per table.

Non-Clustered Index: Maintains a separate structure with pointers. Multiple allowed.

What are different types of SQL Joins?

LEFT JOIN: All records from left table + matching from right.

RIGHT JOIN: All records from right + matching from left.

INNER JOIN: Only matching records from both.

OUTER JOIN: All records with NULLs where no match.

How do you optimize a Stored Procedure?

Use proper indexing.

Avoid cursors.

Use SET NOCOUNT ON.

Analyze execution plan.

Avoid unnecessary computations.

What is Normalization?

Process of structuring a relational database to reduce redundancy:

1NF: Atomic columns.

2NF: Remove partial dependency.

3NF: Remove transitive dependency.

What are Migrations in EF Core?

Track and apply database schema changes.

dotnet ef migrations add InitialCreate

dotnet ef database update

Js & JQuery

Using JQuery Change color on button click

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Change Font Color with jQuery</title>
```

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>

<p id="text">Hello, this is some text!</p>
<button id="changeColor">Change Color</button>

<script>
$(document).ready(function(){
$("#changeColor").click(function(){
$("#text").css("color", "red");
});
});
</script>

</body>
</html>
```

What happens:

When you click the button, the text color of the <p> element changes to red.

JavaScript using Change color on button click

```
<!DOCTYPE html>
<html>
<head>
<title>Change Font Color with JavaScript</title>
</head>
<body>

<p id="text">Hello, this is some text!</p>
<button onclick="changeColor()">Change Color</button>

<script>
function changeColor() {
document.getElementById("text").style.color = "red";
}
</script>

</body>
```


</html>

What happens:

When you click the button, the text color changes to red.

Feature	JavaScript	jQuery
Type	Language	Library built on JS
Syntax	More verbose	Short and simple
DOM Selection	<code>document.getElementById("id")</code>	<code>\$("#id")</code>
Event Handling	<code>element.addEventListener("click", fn)</code>	<code>\$("#id").click(fn)</code>
Animation	Manual with CSS or JS	Built-in with <code>.fadeIn()</code> , <code>.slideUp()</code> , etc.
AJAX Requests	<code>fetch()</code> or <code>XMLHttpRequest</code>	<code>\$.ajax()</code> , <code>\$.get()</code> , etc.
Browser Compatibility	Must handle differences manually	jQuery smooths out browser differences
Performance	Faster (native)	Slightly slower (extra library)

Using JavaScript

```
<button id="btn">Click me</button><p id="text">Hello!</p>
<script>document.getElementById("btn").addEventListener("click", function() {
document.getElementById("text").style.color = "red";
});</script>
```

Using jQuery

```
<button id="btn">Click me</button><p id="text">Hello!</p>
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script>
$("#btn").click(function(){
$("#text").css("color", "red");
});
</script>
```

Modern browsers handle JavaScript much better now, and most jQuery features are built into JavaScript natively.

Angular

What are Angular Decorators?

Functions that attach metadata to classes and methods:

@Component, @NgModule, @Injectable.

What are Directives in Angular?

Custom HTML behavior.

Types:

Structural: *ngIf*, *ngFor*

Attribute: [*ngClass*], [*ngStyle*]

What are Filters in Angular?

Handled via Pipes (Angular doesn't have traditional filters).

{{ amount | currency }}

What are Lifecycle Hooks in Angular?

Methods like:

ngOnInit(): Initialization logic.

ngOnDestroy(): Cleanup code.

What is MVVM Architecture?

Model-View-ViewModel: Separation between UI, data, and logic for better testability and maintainability.

What is the difference between Eager and Lazy Loading?

Eager Loading: Modules load at startup.

Lazy Loading: Load modules only when needed (improves performance).

What are Pipes in Angular?

Transform output in templates.

{{ today | date:'shortDate' }}

What is NgModule?

Defines a context for components and dependencies.

```
@NgModule({  
  declarations: [AppComponent],  
  imports: [BrowserModule]  
})
```

What is NgRx?

Redux-based state management for Angular applications using Store, Actions, and Reducers.

What are Angular Routers?

Used for navigation between views/pages.

`<router-outlet></router-outlet>`

React

What are the components in React?

Reusable pieces of UI.

Functional Components (with hooks)

Class Components (older)

What is state in React?

A built-in object that stores property values that belong to the component.

`const [count, setCount] = useState(0);`

What are props in React?

Props are arguments passed into React components from a parent component.

`<MyComponent name="John" />`

What are the different types of Hooks in React?

`useState()`: Manage local state

`useEffect()`: Side effects

`useContext()`: Context API

`useRef()`, `useMemo()`, `useReducer()`

What is Redux?

A predictable state container for JavaScript apps. Components dispatch actions to update a global store via reducers